PRUEBA TÉCNICA DE SELECCIÓN

Puesto: Desarrollador/a de automatizaciones con IA y sistemas

1. Introducción y objetivo

El objetivo de esta prueba es conocer tu manera de estructurar una solución técnica, tu dominio práctico de JavaScript, bases de datos y APIs, y tu capacidad para trabajar con herramientas de Inteligencia Artificial.

Deberás desarrollar una pequeña aplicación web de gestión de clientes. El ejercicio simula una herramienta interna de la empresa: hasta ahora los clientes y sus facturas se llevaban en un Excel compartido que rellenaban a mano varias personas de la oficina, y hay que pasarlos a una aplicación con base de datos, API y una web.

Buscamos una solución funcional, mantenible y fácil de entender. Se valorarán especialmente la claridad del código, la organización del proyecto, las decisiones técnicas y la capacidad de explicar y modificar la solución posteriormente.

2. Contexto y reglas

- **Lenguaje:** JavaScript o TypeScript con Node.js. El framework es libre (Express, Fastify, Hono…).
- **Base de datos:** PostgreSQL.
- **Arranque:** todo debe levantarse con `docker compose up` desde un repositorio recién clonado.
- **Interfaz:** libre (HTML servido por el backend, React, Vue…). Se evalúa: debe ser clara, coherente, cuidada visualmente y fácil de utilizar.
- **Idioma:** la interfaz en español, para usuarios de España.
- **Inteligencia Artificial:** puedes y debes usarla (ver apartado 6).

Cuando algo no esté especificado, decide tú y déjalo escrito en el README.

3. Datos de partida

Junto a este documento recibirás un fichero CSV (separador `;`, codificación UTF-8).

Fichero `clientes_antiguos.csv`

El Excel de clientes y facturas que se usaba hasta ahora, con una fila por factura. Columnas y valores de la primera fila:

Columna	Ejemplo	Columna	Ejemplo
empresa	Construcciones Ebro SL	pais	España
cif	B50123456	comercial	Elena Sanz
contacto	Marta Gil	estado	Activo
email	marta.gil@construccionesebro.es	fecha_alta	14/01/2025
telefono	+34 976 123 456	factura	F-2026-0101
direccion	Calle Alfonso I, 12	fecha_factura	03/09/2026
cp	50003	base_imponible	1.250
ciudad	Zaragoza	iva	21%

4. Descripción del desafío

Requisito A - Base de datos

Crea una base de datos PostgreSQL con los clientes y las facturas del fichero `clientes_antiguos.csv`. Cada cliente deberá contener, como mínimo:

- Empresa o razón social y CIF.
- Contactos: nombre, correo electrónico y teléfono.
- Estado (Activo / Inactivo), ciudad y país.
- Comercial asignado y fecha de alta.
- Sus facturas: fecha, base imponible, IVA y total.
- Facturación acumulada: la suma de las bases imponibles de sus facturas, sin IVA.
- Fecha de última modificación.

La base de datos también debe guardar el histórico de cambios (requisito D) y lo necesario para que la API sea segura (requisito B).

Requisito B - API REST

Además de la interfaz web, la aplicación deberá exponer una API. La API debe ser segura: nadie sin acceso debe poder consultar ni modificar los datos.

Método	Ruta	Objetivo
GET	`/api/clientes`	Listado paginado; búsqueda por empresa, contacto, CIF o email; filtros por estado, país y comercial
GET	`/api/clientes/{id}`	Cliente con sus contactos, facturas y facturación acumulada
POST	`/api/clientes`	Crear un nuevo cliente
PUT	`/api/clientes/{id}`	Actualizar un cliente existente
DELETE	`/api/clientes/{id}`	Eliminar un cliente
GET	`/api/clientes/{id}/historico`	Histórico de cambios del cliente

Incluye una colección de ejemplos para probar la API (`curl`, `.http` o Postman).

Requisito C - Interfaz web

1. Vista de listado

- Mostrar los clientes en una tabla o cuadrícula limpia con sus datos esenciales.
- Incluir paginación.
- Permitir búsqueda por empresa, contacto, CIF o correo electrónico.
- Permitir filtros por estado, país y comercial asignado.
- Incluir acciones para consultar o editar cada cliente.

2. Gestión de clientes

- Crear, editar y eliminar clientes.
- Ficha del cliente con sus contactos, facturas, facturación acumulada e histórico de cambios.

Requisito D - Histórico de modificaciones

Cuando se modifique un cliente o uno de sus contactos, la aplicación deberá registrar un histórico de cambios. Como mínimo se guardarán: registro modificado, fecha y hora, campo modificado, valor anterior, valor nuevo y usuario que hizo el cambio. Debe existir una forma sencilla de consultar el histórico de un cliente desde la aplicación.

Requisito E - Puesta en marcha y calidad

- `docker compose up` levanta base de datos, API y web. Un fichero `.env.example` documenta cada variable.
- README con: cómo arrancar, cómo probar la API, decisiones técnicas y lo que dejarías para más adelante.
- Tests automáticos que se ejecuten con un solo comando.

5. Criterios de evaluación

Aspecto	Qué valoramos	Peso orientativo
Estructura y calidad	Organización, legibilidad, separación de responsabilidades y mantenibilidad.	35 %
Funcionamiento real	Ejecución local estable con Docker y cumplimiento del flujo solicitado.	25 %
Datos y robustez	PostgreSQL, persistencia, validaciones y tratamiento de errores.	15 %
Experiencia de usuario	Interfaz clara, coherente, cuidada visualmente y fácil de utilizar.	10 %
Defensa técnica	Comprensión, explicación y capacidad de modificar la solución.	15 %

La defensa técnica se valora en la revisión posterior (apartado 8).

6. Uso de herramientas de Inteligencia Artificial

Puedes y debes utilizar herramientas de IA (Claude Code, Cursor, ChatGPT, Copilot u otras) durante la prueba. Su uso no supone ninguna penalización: programar con IA es parte del puesto.

Deberás comprender, explicar y ser capaz de modificar todo el código entregado. Durante la revisión técnica podrán solicitarse pequeños cambios sobre la aplicación en tiempo real.

7. Entrega

- Sube el proyecto a un repositorio **público de GitHub**, con el historial de commits tal cual, y envíanos el enlace.
- Fichero `README.md` en la raíz del repositorio.
- Incluye los ficheros de conversación con la IA que hayas usado, o un resumen de los prompts principales. No penaliza: nos interesa ver cómo la diriges.
- No incluyas `node_modules`, ficheros `.env` ni claves.
- Plazo máximo de entrega: 72 horas desde la recepción de este documento.

8. Revisión técnica posterior

Tras la entrega se realizará una revisión técnica breve (aproximadamente 20-30 minutos). Deberás explicar la estructura de la aplicación, las decisiones tomadas y el funcionamiento de las partes principales.

Durante esta revisión podrá solicitarse una pequeña modificación funcional sobre el proyecto entregado. El objetivo no es valorar la velocidad de escritura, sino comprobar la comprensión del código, la capacidad de localizar los puntos que deben modificarse y la forma de razonar ante un cambio.

9. Criterio libre

Cualquier detalle técnico no especificado queda a tu criterio: librerías, estructura de carpetas, patrones y enfoques, siempre que la solución se pueda arrancar y probar en local y que expliques tus decisiones en el README.

No buscamos la solución más compleja. Preferimos una aplicación sencilla, sólida, bien estructurada y comprensible frente a una implementación sobredimensionada.

¡Mucho éxito con el ejercicio!